

DSA - Row 12

I. 1) Queue on circular array

10 items in the queue

data[2] → data[11]

current capacity = 11.

where will the next elem. be inserted?

(ADT) Queue = container in which access is restricted to FRONT and REAR.
(pop) (push)

(DS) Circular array: Queue:

maxElem: Int

cap: Int

elems: TElem[]

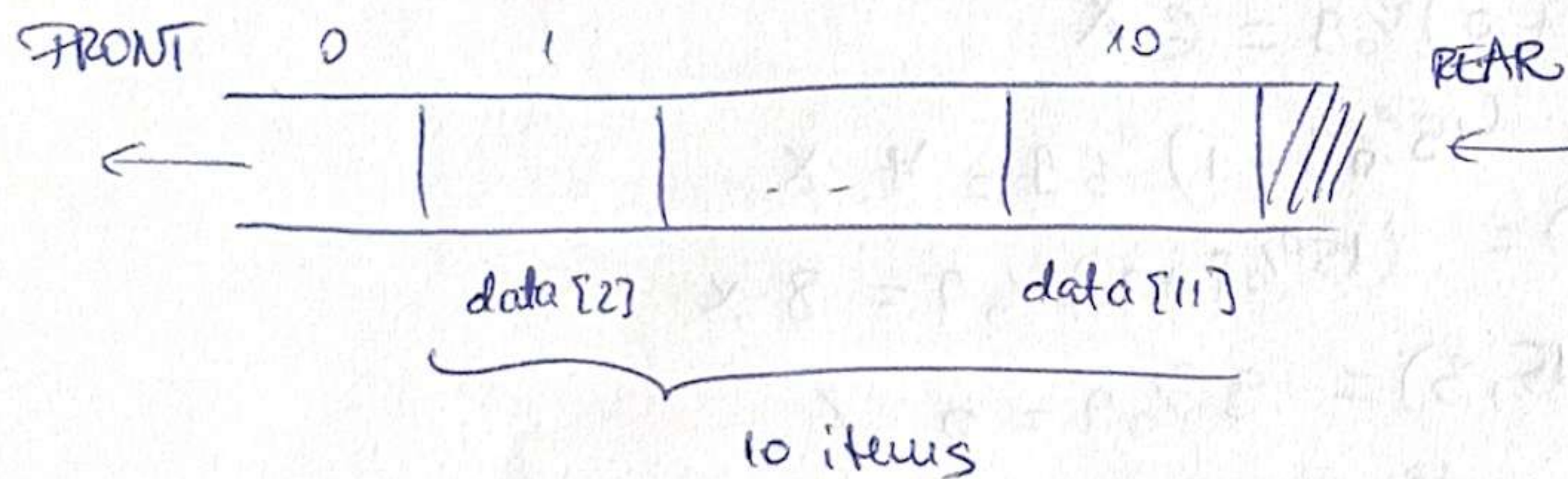
front: Int

rear: Int

+ isFull()

Convention for circular array: front = pos. of the first elem.
rear = pos. AFTER the last elem.

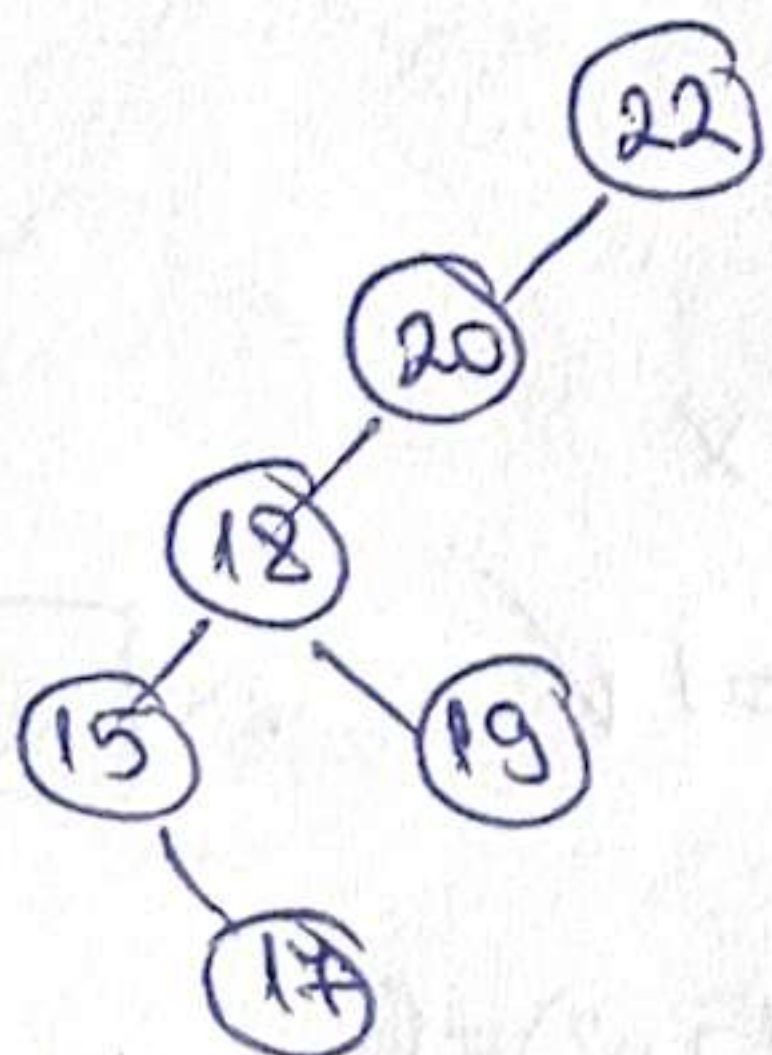
⇒ push, pop, front in $\Theta(1)$.



BUT array has capacity 11 ⇒ $\nexists$ position 12

circular array ⇒ data[i] ⇒ [a)

2) BST → all the insertion orders that could have produced it.



constraints:

22 before 20

20 before 18

18 before 15 and 19

15 before 17

BST = BT + extra property (left ≤ root ≤ right)

- can have unfiled levels

- each node has max 2 children

- Full BST when all nodes have 2 children (except leaves)

- COMPLETE BST when all leaves are on the same level

- parent must be inserted before its children

22 20 18 19 15 17 / 22 20 18 15 17 19 ⇒ 3 possible orders

3) HT, open addressing, linear probing, div. method
no elem. was removed
in which order to insert elements?

0	1	2	3	4	5	6	7	8
15	23	11			59	51	79	44

probe sequence: $h(k, i)$, $i = \overline{0, m-1}$

linear probing: $h(k, i) = (h'(k) + i) \% m$

$$h'(k) = k \% m.$$

$$m = 9, i = \overline{0, 8}$$

a) 59, 44, 51, 11, 79, 15, 23.

$$h(59, 0) = (59 \% 9 + 0) \% 9 = 5 \% 9 = 5 \checkmark$$

$$h(44, 0) = (44 \% 9 + 0) \% 9 = 8 \% 9 = 8 \checkmark$$

$$h(51, 0) = (51 \% 9 + 0) \% 9 = 6 \% 9 = 6 \checkmark$$

$$h(11, 0) = (11 \% 9 + 0) \% 9 = 2 \checkmark$$

$$h(79, 0) = (79 \% 9 + 0) \% 9 = 7 \checkmark$$

$$h(15, 0) = (15 \% 9 + 0) \% 9 = 6 \times$$

$$\hookrightarrow h(15, 1) = (15 \% 9 + 1) \% 9 = 7 \times$$

$$\hookrightarrow h(15, 2) = (15 \% 9 + 2) \% 9 = 8 \times$$

$$\hookrightarrow h(15, 3) = 9 \% 9 = 0 \checkmark$$

$$h(23, 0) = (23 \% 9 + 0) \% 9 = 5 \times$$

$$\hookrightarrow \cancel{h(23, 1) = 6 \% 9 = 6 \times}$$

$$h(23, 1) = (23 \% 9 + 1) \% 9 = 6 \% 9 = 6 \times$$

$$\hookrightarrow h(23, 2) = 7 \% 9 = 7 \times$$

$$\hookrightarrow h(23, 3) = 8 \% 9 = 8 \times$$

$$\hookrightarrow h(23, 4) = 9 \% 9 = 0 \times$$

$$\hookrightarrow h(23, 5) = 10 \% 9 = 1 \checkmark$$

$\Rightarrow \boxed{a}$

$$b) h(11, 0) = 2$$

$$h(23, 0) = 5 \times$$

$$c) h(44, 0) = 8 \checkmark$$

$$h(51, 0) = 6 \checkmark$$

$$h(79, 0) = 7 \checkmark$$

$$d) h(15, 0) = 6 \checkmark$$

$$h(59, 0) = 5 \checkmark$$

$$\boxed{e) h(51, 0) = 6 \checkmark}$$

$$h(59, 0) = 5 \checkmark$$

$$h(44, 0) = 8 \checkmark$$

$$h(79, 0) = 7 \checkmark$$

$$h(11, 0) = 2$$

$$h(51, 1) = 6 \checkmark$$

$$h(15, 0) = 6 \rightarrow h(15, 1) = 7$$

$$h(15, 0) = 6$$

$$1 \Rightarrow 7$$

$$2 \Rightarrow 8$$

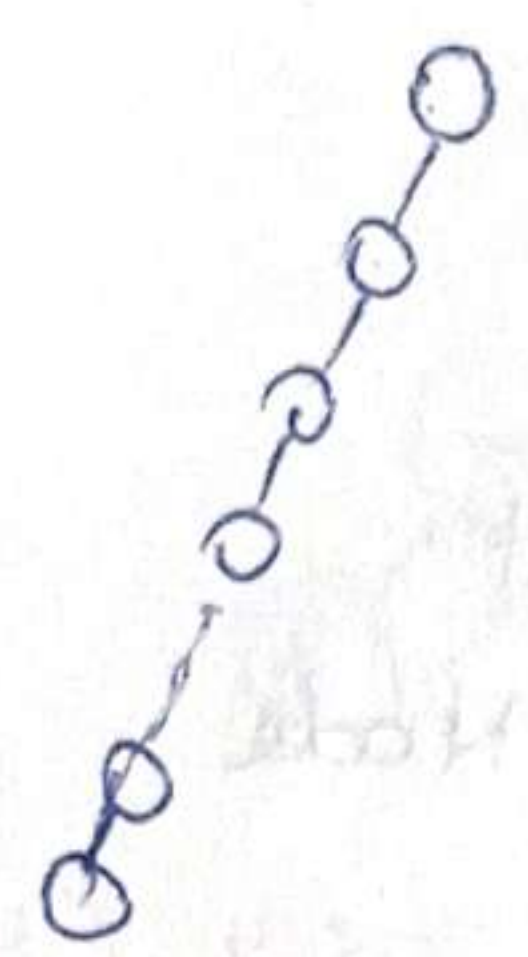
$$3 \Rightarrow 0 \checkmark$$

$$h(23, 0) \Rightarrow 5 \checkmark$$

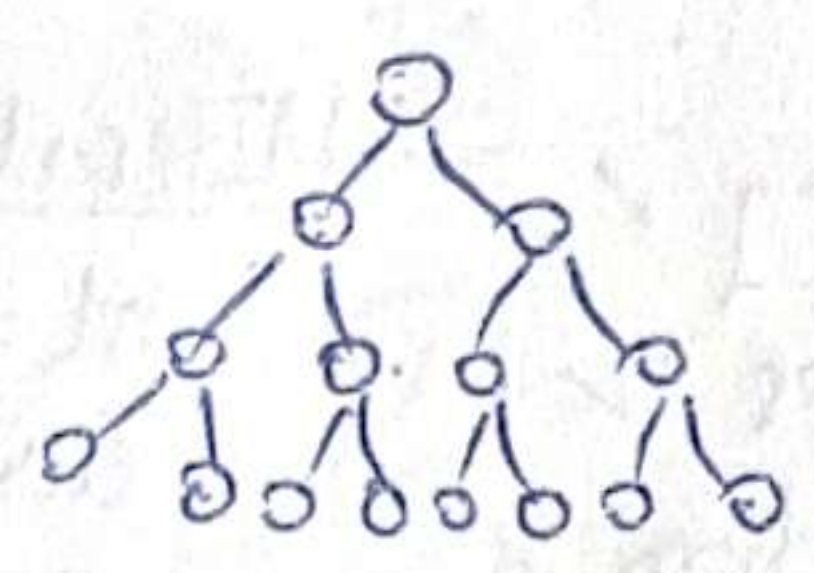
$$\hookrightarrow h(15, 1) = 8 \times$$

1) min/max depth of a BT with 15 nodes?

depth: root
↓
node



max = 14
(when the BT is a chain)



min = 3
(when the BT is as full/complete as possible)

II. 1) MAX Binary heap : add: 45, 56, 78, 23, 11, 54, 88

→ heap before/after adding 78 $\rightarrow O(\log_2 n)$

→ final heap → tree form
→ array form (cap = 10)

→ remove 2 elems. $\rightarrow O(\log_2 n)$

max 2 children

Binary Heap = single + complete (BT) + structure + property

↳ every level is filled, except maybe the last one.

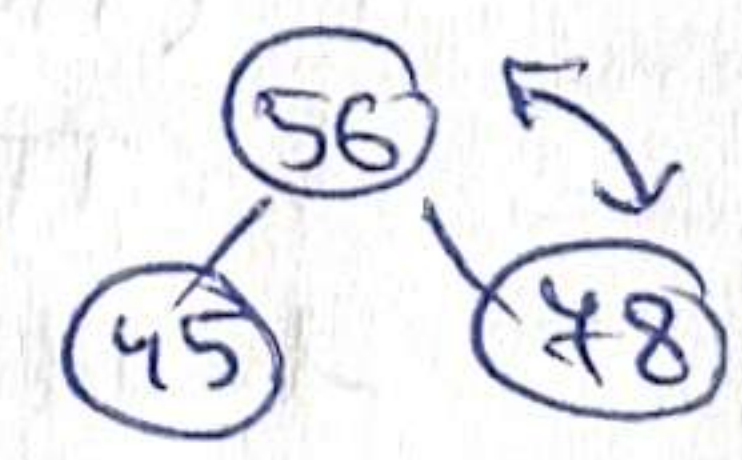
↳ structure (no holes)

↳ property (max/min)

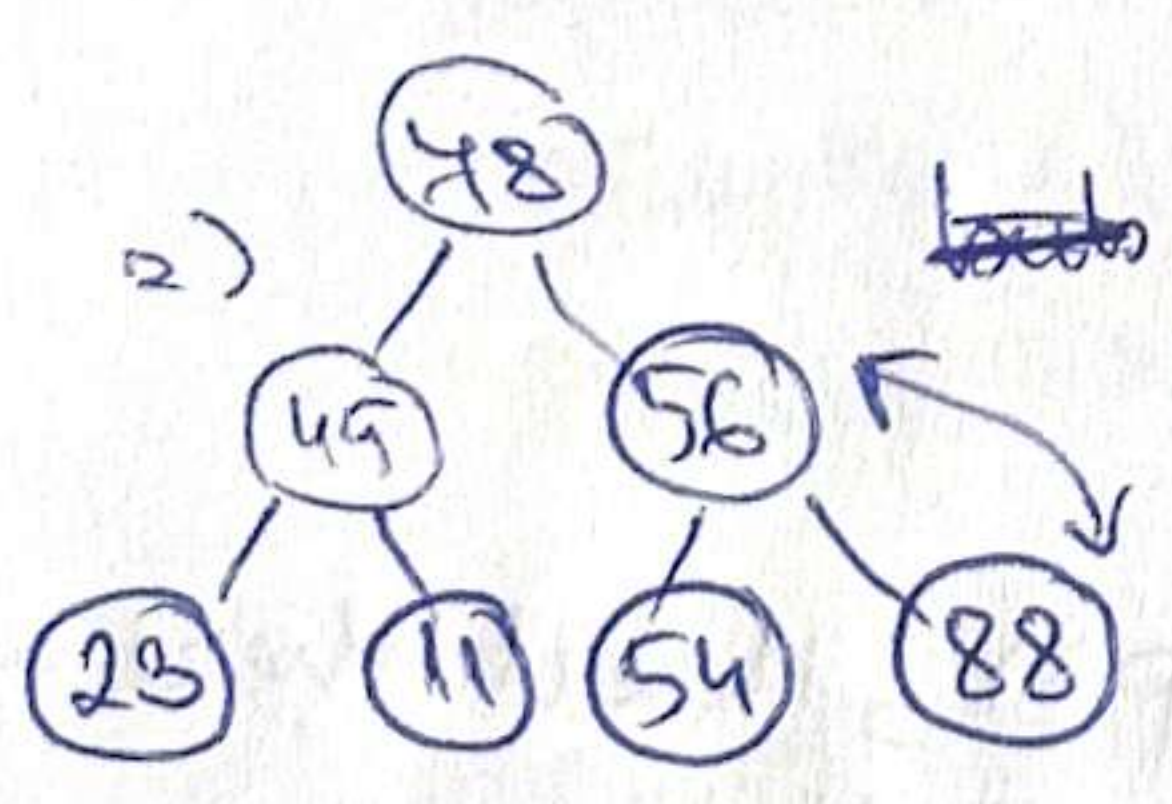
↳ efficient repres. for Priority Queue



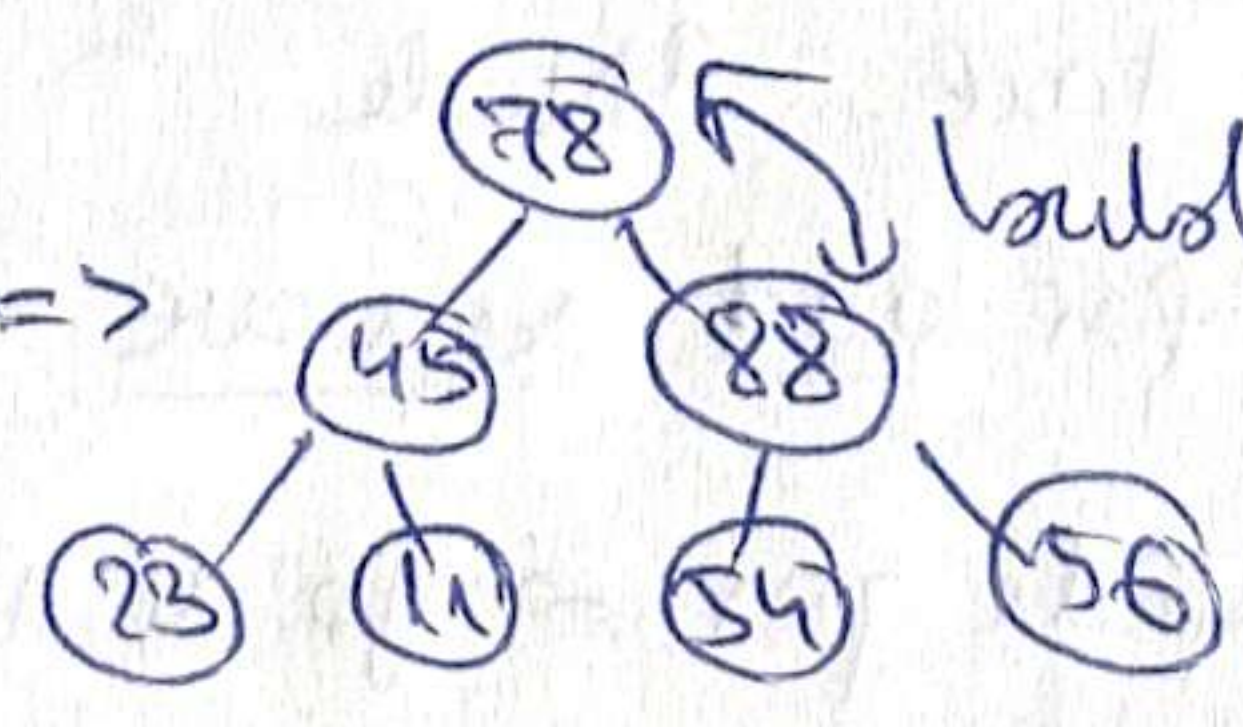
bubble-up $\Rightarrow$



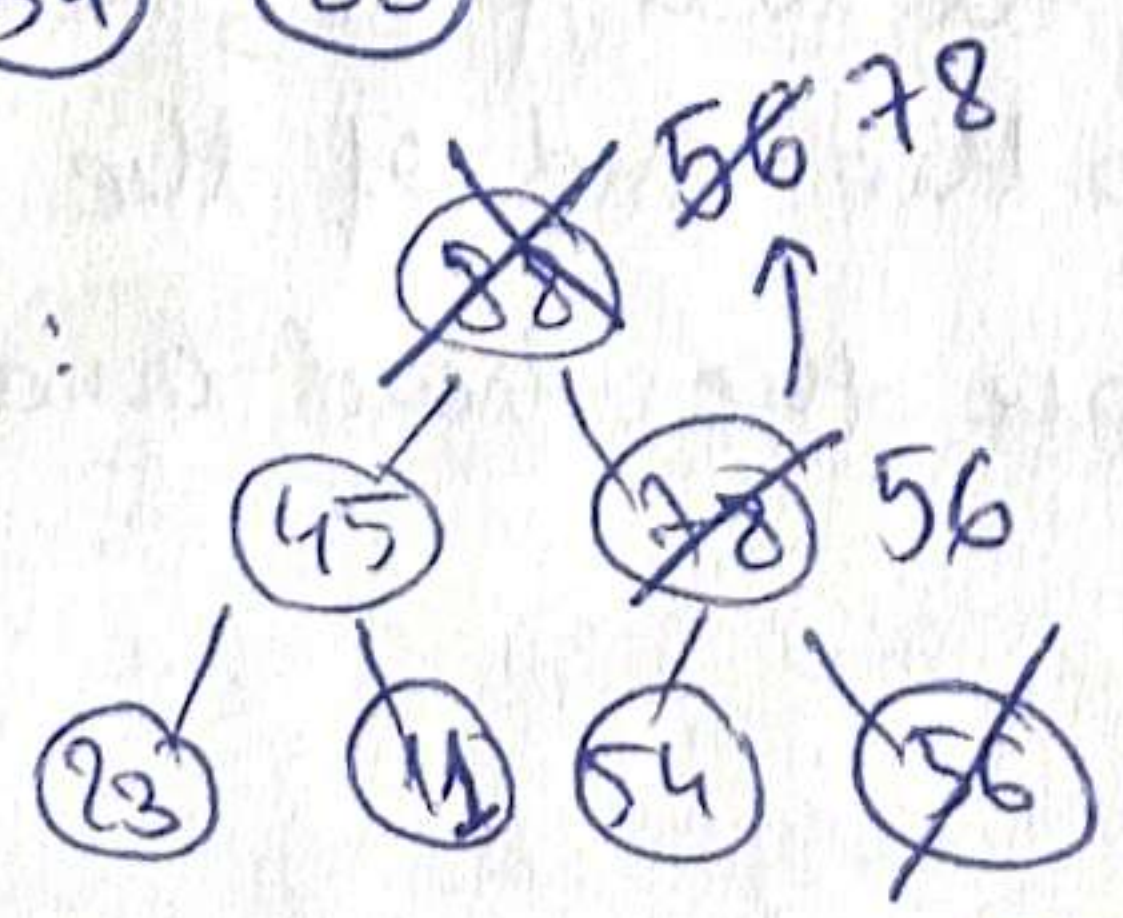
bubble-up



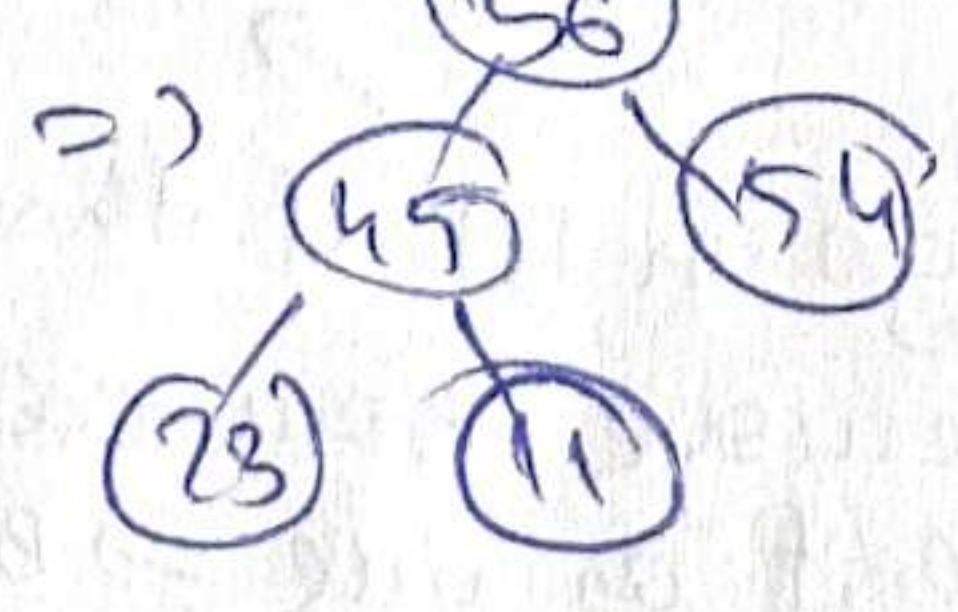
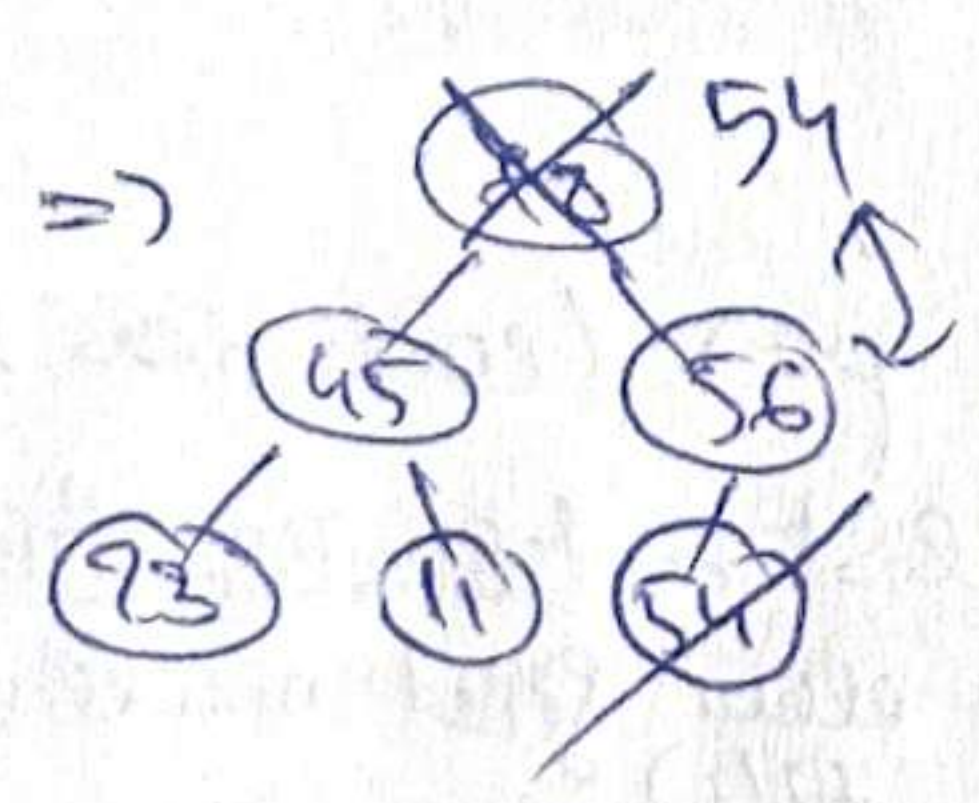
bubble up $\Rightarrow$



$\Rightarrow$ Time:



$\Rightarrow$ array form: [88, 45, 78, 23, 11, 54, 56]



0 1 2 3 4 5 6 7 8 9
 $\Rightarrow$ [56, 45, 54, 23, 11, ---]

2) Linked HT = HT with sep. chaining ⊕ iterator in $\Theta(1)$ op. that returns elems. in insertion order.

a) Linked HT + Iterator repres.

Node:

key: TKey

next: $\uparrow$ Node

HT:

T: $\uparrow$ Node[] ✓

m: Integer ✓

h: TFunction ✓

HTIterator:

ht: HT

currentPos: Int

currentNode: $\uparrow$ Node

Linked HT: DS which has a predictable iteration order (the order in which elems. were inserted)

! we need a DLL because the only operation that cannot be implemented in $\Theta(1)$ is remove if we were to use a SL. (because when we remove we need the elem. before it and we need to search $\Rightarrow \Theta(n)$).

Node:

info: TKey

nextH: $\uparrow$ Node $\rightarrow$ collision

nextL: $\uparrow$ Node } insertion-order list

prevL: $\uparrow$ Node }

Linked HT:

m: Integer

T: ($\uparrow$ Node)[]

h: TFunction

head: $\uparrow$ Node

tail: $\uparrow$ Node

b) Bc, we for add, remove

add $\rightarrow$ compute pos $\rightarrow$ go in HT at that pos, insert there $\rightarrow$
 $\rightarrow$ insert that new node to the end of the insertion-order list as well $\rightarrow$ we have the head and tail $\rightarrow$
 $\rightarrow$ insert at tail
 $\Rightarrow$ Bc: $\Theta(1)$
 $\Rightarrow$ wc: $\Theta(1)$

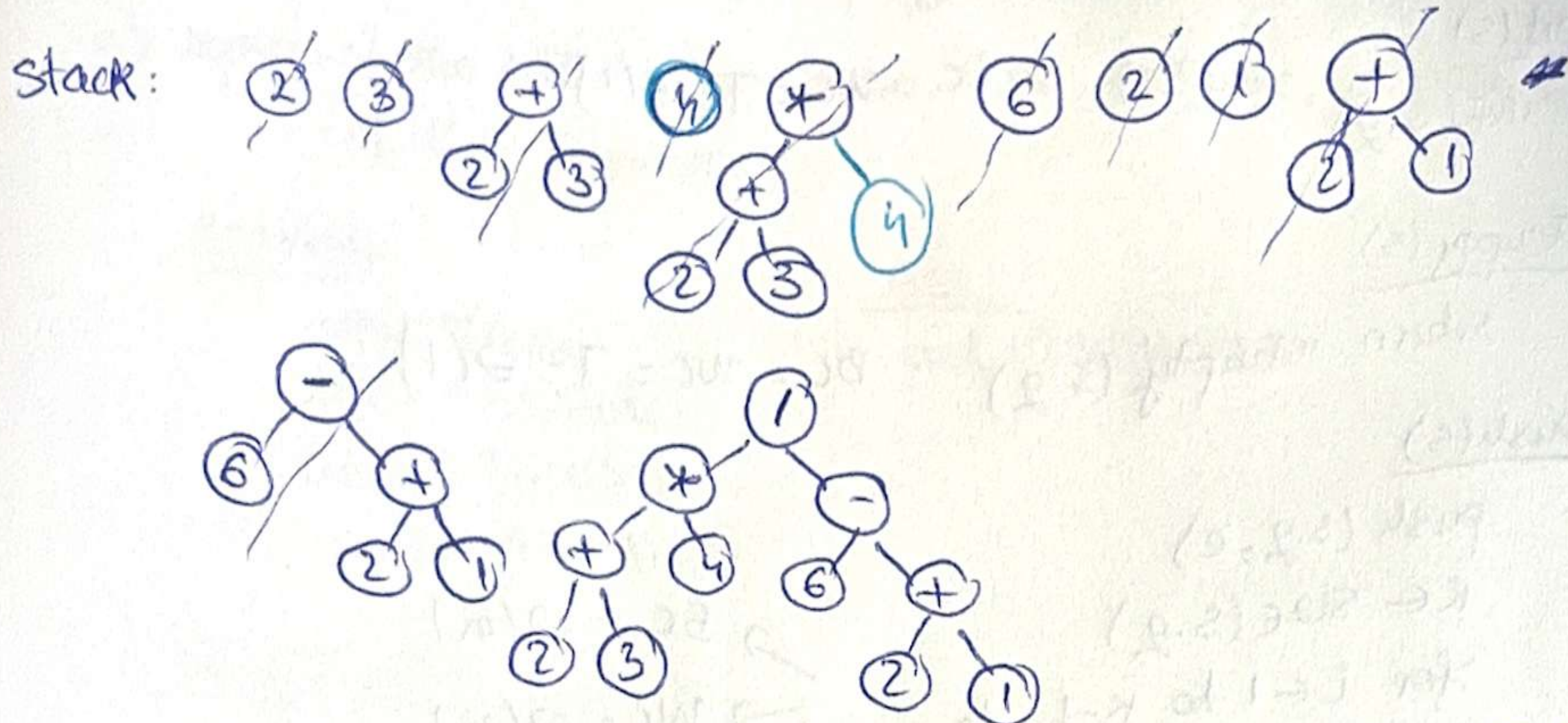
remove $\rightarrow$ compute pos $\rightarrow$ go in HT at that pos (collision list)

$\rightarrow$ traverse coll. list. $\rightarrow$ if pos found $\rightarrow$ remove from coll. list $\rightarrow$ remove from insertion-order list as well $\rightarrow$ Bc: elem. first in collision list $\Theta(1)$
 $\rightarrow$ if not found $\rightarrow$ STOP.
Coll. list $\Rightarrow$ traverse all list $\Rightarrow \Theta(n)$

$\Theta(1)$ always

be DLL and we know prevL

3) BT, postfix: 2 3 + 4 * 6 2 1 + - /



The algorithm uses a stack as an internal structure. If an operand (2, 3, 4, 6, ...) is encountered, ~~then~~ it's pushed on the stack. if an operator (+, -, /, *) is encountered, we pop (the last 2) operands from the stack. The one we pop first goes on the right of the operator, the other one on the left.

iii. Extended Queue ADT $\sim$ Queue ADT + $\frac{\text{size}(Q)}{\text{return mr. of elems. in the queue}}$
Stack over an Extended Queue?

a) specs. for the op. of the Extended Queue

Q11) {
 init(Q)
 push(Q, e) // inserts at rear
 pop(Q) // removes the front
 top(Q) // remove + return the front
 isEmpty(Q) // tests if the Q is empty
 size(Q) // returns nr. of elems.

b) repres. of stack over Extended Queue

Stack:

Q: Extended Queue

c) How DS should work for push/pop?

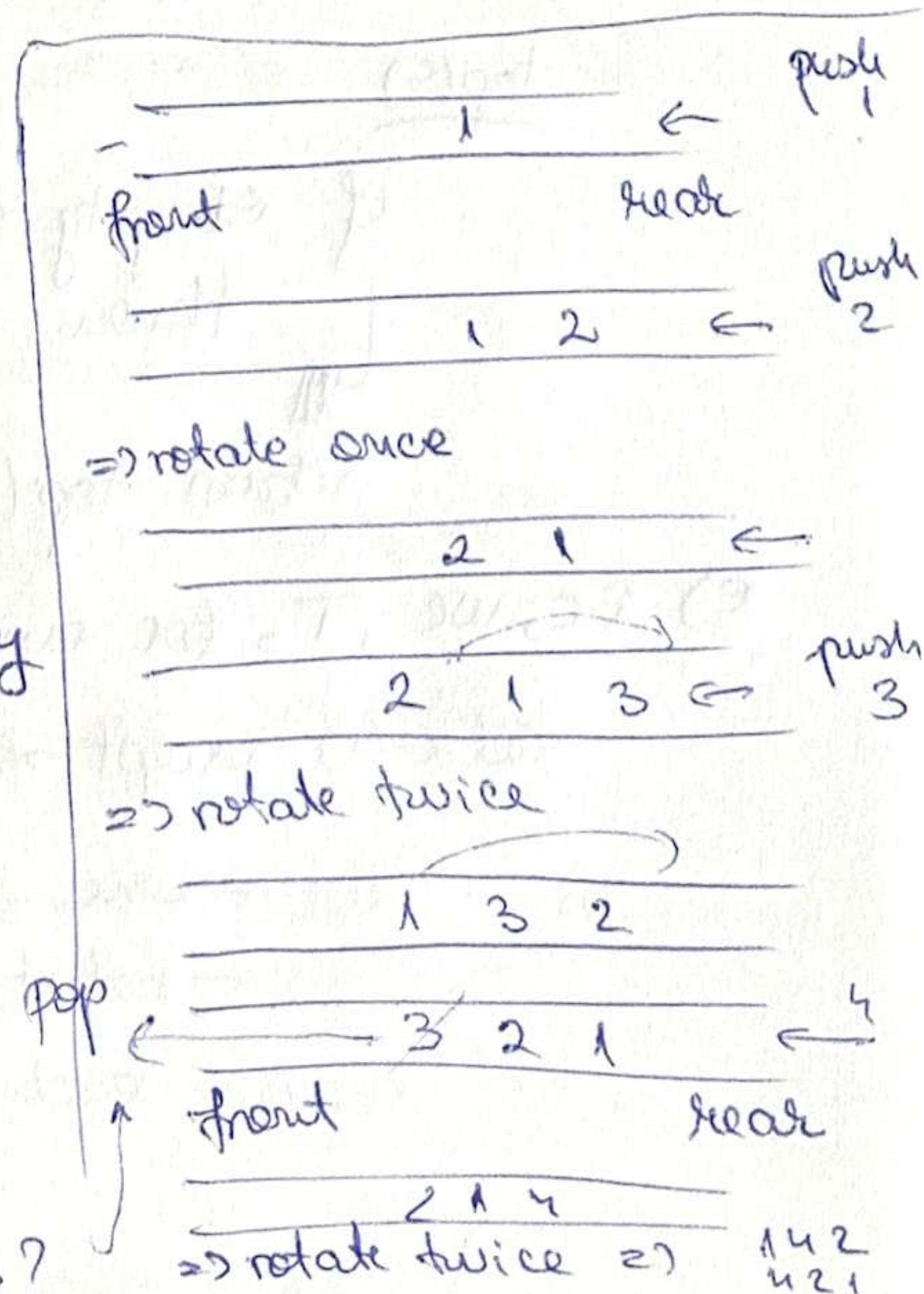
Content of int. repres $\rightarrow$ after pushing 1, 2, 3 to an empty stack

$\rightarrow$ after popping 1 elem $\Rightarrow$ 2 1

$\rightarrow$ after adding 4 $\Rightarrow$ 4 2 1

$\rightarrow$ after popping 1 elem. $\Rightarrow$ 2 1

Queue $\Rightarrow$ FIFO } $\Rightarrow$ after inserting a new element, rotate all older elements behind it.
 Stack $\Rightarrow$ LIFO



DSA - row 6 II.

function init(s)

d) implement all stack op., using op. of the Extended Queue.

init(s)
init(s.g)

$\Theta(1) \rightarrow BC = WC = T = \Theta(1)$

isEmpty(s)

return isEmpty(s.g)

$BC = WC = T = \Theta(1)$

push(s)

push(s.g, e)

$k \leftarrow \text{size}(s.g)$

for $i \leftarrow 1$ to $k-1$ do

$x \leftarrow \text{pop}(s.g)$

push(s.g, x)

$BC = \Theta(n)$
 $WC = \Theta(n)$
 $T = \Theta(n)$

pop(s)

if isEmpty(s.g) then
throw...

$\Theta(1)$

return pop(s.g)

top(s)

if isEmpty(s.g) then
throw...

$BC = WC = T = \Theta(1)$

return top(s.g)

e) BC, WC, T , for every stack op., assuming all EQ op. are $\Theta(1)$

all $\Theta(1)$ except for push $\rightarrow \Theta(n)$ because:

- one enqueue: $\Theta(1)$

- rotate (size-1) elems.

- each rotation: pop $\Theta(1)$, $\therefore$, $\Theta(1)$ work/rot.
push $\Theta(1)$

- $n-1$ rotations $\Rightarrow$ total work: n .

4) Iterator for a BST with $\Theta(1)$ extra storage.

a) Repres. of the BST (Linked repres. with dyn. alloc, w parent)
- of the Iterator

BSTNode

info: TElem

left: $\uparrow$ BSTNode

right: $\uparrow$ BSTNode

parent: $\uparrow$ BSTNode

BST:

root: $\uparrow$ BSTNode

BSTIterator:

tree: $\uparrow$ BST

current: $\uparrow$ BSTNode

b) Specify ABI Iterator

operations:

first(it)

next(it)

getCurrent(it)

valid(it)

(inorder iterator)

c) complexity of next (BC, WC)

BC: successor is found immediately (current only has a right child) $\Rightarrow \Theta(1)$

OR

we: current is a left child of its parent.

WC: go ~~down~~ right down, then left in a long path

OR

climb up through many parents $\Rightarrow \Theta(h)$

$\Theta(1)$ extra storage bc we only store - current

- child

- parent

b) Implement iterator op.

subalg first(it) is

```
if it.bst.root = Nil then
    it.current ← Nil
else
    it.current ← min(it.bst.root)
```

subalg min(node) is

```
while node.left ≠ Nil
    node ← node.left
min ← node
```

BC: $\Theta(1)$

WC: $\Theta(h)$

function valid(it) .

```

if it.current != Nil then
    valid ← True
else
    valid ← False

```

function getCurrent(it)

```

if not valid(it) then
    @throw exception
    ↵
getCurrent ← it.current.info

```

Subalg next(it) is

```

if not valid(it) then
    @throw ...
    ↵
it.current ← successor(it.current)

```

Subalg successor(mode) is

```

if mode.right != Nil then
    successor ← min(mode.right)
    return
    ↵
p ← mode.parent
while p != Nil and
    mode = p.right ex
    mode ← p
    p ← p.parent

```

